COLEXIO VIVAS S.L.	RAMA:	Computing	CICLO:	MAP	
	MÓDULO	Service and processes programming		CURSO:	2º
	PROTOCOLO:	Reading	AVAL:	DATA:	
	Autor	Francisco Bellas Aláez (Curro)			

Archivos, delegados y lambdas.

Acceso a Archivos en C#

Los archivos son flujos o secuencias de bytes de E/S.

En .Net, las clases que nos permiten manejar la E/S de datos se encuentra en el espacio de nombres denominado **System.IO** (de Input/Output).

Para el manejo de archivos conviene especificar la importación de este espacio de nombres si no se quiere escribir el IO siempre:

```
using System.IO;
```

Las clases de manejo de archivos heredan de las clases abstractas **Stream, TextReader** o **TextWriter**. Un Stream es una secuencia o flujo de bytes de salida o de entrada (archivos binarios) y TextReader y TextWriter permiten el manejo de archivos de texto.

Un ejemplo claro de esto último lo tenéis con la **Consola** donde se le envía un flujo de caracteres mediante Write o WriteLine y la consola se encarga de mostrarlos. La consola contiene objetos **TextReader** y **TextWriter** para poder hacer estas tareas. Puedes acceder a ellos mediante las propiedades **In** y **Out** de Console.

En este tema vamos a ver clases que heredan de las anteriormente comentadas y que nos van a permitir el manejo de archivos y directorios.

Se debe tener presente siempre que aunque se trabajen con distintos "tipos de archivos" (binario/texto), en el fondo esto no es más que un convenio cómodo para los programadores. **Un archivo son siempre datos binarios** (ceros y unos) que tienen distinto significado según nos interese. Algo similar a los tipos de datos al declarar una variable.

Por último hay que tener en cuenta que **las operaciones sobre archivos se realizan sobre un buffer intermedio**, no directamente sobre el archivo físico. Así al estar modificando un archivo **puede no reflejarse en el disco duro** hasta que se haga el volcado del buffer al disco duro mediante la instrucción correspondiente o **cerrando el archivo.**

Archivos de Texto

La clase StreamWriter (Text Files) y uso de variables de entorno.

Clase que permite realizar operaciones de escritura sobre un **archivo de texto**. Su utilización habitual consta de 3 fases:

- Instanciar el objeto mediante un constructor: Aquí se le indica tanto el nombre del archivo con el que se va a trabajar como si se va a reescribir el archivo o si se va a añadir texto al final del mismo (append).
- Escritura del texto mediante *Write* o *WriteLine*
- Cierre del archivo mediante *Close*. Esto además vuelca el contenido del buffer al archivo físico.

Es necesario recordar que el acceso a archivos de texto es **secuencial**.

Ejemplo (El System.IO está en el using):

```
string directory;
string separator;
string fullName;
StreamWriter s;

// If only use Windows, you don't need to do this
if (Environment.OSVersion.Platform == PlatformID.Unix)
{
    directory = Environment.GetEnvironmentVariable("HOME");
    separator = "/";
}
else
{
    directory = Environment.GetEnvironmentVariable("homepath");
    separator = "\\";
}
fullName = directory + separator + "testfile.txt";

// Open File
s = new StreamWriter(fullName);
```

```
// Write File
for (int i = 1; i <= 10; i++)
{
    s.Write("{0,-2}", i);
}
s.WriteLine();


for (int i = 1; i <= 10; i++)
{
    s.WriteLine($"Line :{i,3}");
}

// Close File. Needs try-catch.
s.Close();
```

Se puede ver una forma de detectar plataforma por si estamos realizando una clase que sea multiplataforma mediante **Environment.OSVersion.Platform**.

Mediante **GetEnvironmentVariables** obtenemos el contenido de la **variable de entorno** deseada. No importa si se escribe en mayúsculas o minúsculas y no se ponen los % que la rodean habitualmente en Windows ni el $() de sistemas Unix.

Para usar variables de entorno existe también el método **Environment.ExpandEnvironmentVariables** que se puede usar de la siguiente forma:

```
string query = "My user path is %HOMEDRIVE%%HOMEPATH% " +
    "and my architechture is %PROCESSOR_ARCHITECTURE%";
string str = Environment.ExpandEnvironmentVariables(query);
Console.WriteLine(str);
```

Si quisiéramos agregar texto a un archivo (**append**), el constructor sobrecargado usado debería ser:

```
s = new StreamWriter(fullName, true);
s.WriteLine("Appending text to file");
s.Close();
```

Si se pone como segundo parámetro **true** en el constructor, la acción es la de añadir texto al final (equivalente al append clásico). Si se pone a false o no se pone dicho parámetro, sobreescribe el archivo o lo crea si no existía.

Si en algún momento se desea volcar la información del buffer al archivo sin cerrar el StreamWriter, se puede hacer mediante el método **Flush().**

La clase StreamReader

Esta clase permite realizar lectura de datos de texto desde un archivo. Presenta al igual que en el caso anterior distintos métodos para realizar la lectura.

Al igual que en el caso anterior, se inicializa con el constructor pasándole como parámetro el nombre del archivo a leer.

En el caso de que un archivo no exista saltará una excepción **FileNotFoundException**.

Métodos:

ReadLine(): Devuelve una cadena que contiene una línea completa.

ReadToEnd(): Devuelve el texto desde la posición actual hasta el final del archivo.

Ejemplos: Lectura de un archivo línea a línea mediante un bucle:

```
string line;
StreamReader sr;
sr = new StreamReader(fullName);
line = sr.ReadLine();
while (line != null)
{
    Console.WriteLine(line);
    line = sr.ReadLine();
}
sr.Close();
```

Lectura de un archivo de texto entero y mostrarlo.

```
sr = new StreamReader(fullName);
Console.WriteLine(sr.ReadToEnd());
sr.Close();
```

Excepciones y cierre automático (using)

La lectura y escritura de datos en archivos evidentemente genera excepciones. La **excepción** más habitual en este caso es que el directorio/archivo donde se quiere trabajar no exista o no haya permisos en el mismo. Para ello se puede utilizar el clásico try-catch-finally.

Para una revisión sobre excepciones se puede ver el enlace:

https://msdn.microsoft.com/es-es/library/system.io.ioexception(v=vs.110).aspx

Si no se desea hacer nada en caso de que salte una excepción pero se quiere realizar un cierre automático se puede reducir la sentencia a un try-finally. Es decir, evitando el bloque catch y cerrando el archivo en el finally si es distinto de null.

Sin embargo existe en C# una forma de escribir este bloque más cómoda mediante la sentencia **using**. Esta es similar al conocido try-with-resources de Java, pero sin la captura de excepciones, solo cierra el archivo automáticamente.

Tomando el último ejemplo, quedaría de la siguiente forma:

```
using (sr = new StreamReader(fullName))
{
    Console.WriteLine(sr.ReadToEnd());
}
```

En este caso el recurso que se abre dentro del using se cierra automáticamente con las comprobaciones pertinentes.

Este ejemplo que se usa con clases de archivos, realmente va a servir para cualquier recurso que necesite cerrarse (conexiones de red, bases de datos,…). La condición es que la clase implemente la interfaz **IDisposable**.

Si se desea hacer el control de excepciones simplemente se mete en un sentencia try el bloque anterior.

Ejemplo:

```
try
{
     using (StreamReader sr = new StreamReader("MyFile.txt"))
     {
         Console.WriteLine(sr.ReadToEnd());
     }
 }
 catch (FileNotFoundException e)
 {
     Console.WriteLine($"The file was not found: '{e}'");
 }
 catch (DirectoryNotFoundException e)
 {
     Console.WriteLine($"The directory was not found: '{e}'");
 }
 catch (IOException e)
 {
     Console.WriteLine($"The file could not be opened: '{e}'");
 }
```

Directorios importantes y sus variables de entorno (Windows)

A continuación se relatan algunas variables de entorno que apuntan a directorios de interés. Alguna de ellas en realidad tiene una descripción más amplia según el sistema informático donde esté funcionando, pero aquí se simplifica para el uso que le vamos a dar.

User data: APPDATA

Aquí las aplicaciones guardan, habitualmente en un subdirectorio, la información de configuración de cada usuario. Es un directorio distintos según el usuario.

Global Data: PROGRAMDATA or ALLUSERSPROFILE

Aquí las aplicaciones guardan, habitualmente en un subdirectorio, la información de configuración global de la aplicación que es común a todos los usuarios. Los servicios también pueden guardar aquí la configuración y otra información. Es un directorio único en el ordenador.

User directory: HOMEPATH

Directorio base del usuario. Es un directorio distintos según el usuario.

User drive: HOMEDRIVE

Unidad de disco donde se encuentra el directorio de usuario.

User path: USERPROFILE

Ruta completa del directorio de usuario. Es equivalente a %HOMEDRIVE% %HOMEPATH%

System directory: WINDIR or SYSTEMROOT

Directorio donde se encuentra Windows.

Manejo e información de archivos y directorios

Archivos: Clases File y FileInfo

Ambas clases sirven para el manejo de archivos según una serie de métodos.

File

No hace falta instanciarla, pues contiene métodos estáticos.

Los métodos más comunes de esta clase son:

Exist(filename): Devuelve true si existe cierto fichero.

Copy, Move, Delete: Copia, mueve y borra archivos respectivamente. Ver la ayuda pues son sencillas de manejar.

Ejemplo:

```
StreamReader sr;
StreamWriter sw;
string fullName;

Console.WriteLine("Please, introduce file name with full path:");
fullName = Console.ReadLine();
if (File.Exists(fullName))
{
    using (sr = new StreamReader(fullName))
    {
        Console.WriteLine(sr.ReadToEnd());
    }
}

else
{
    using (sw = new StreamWriter(fullName))
    {
        sw.WriteLine("This is");
        sw.WriteLine("a new file");
    }
}
```

Dispone también delas funciones **WriteAllText**, **AppendAllText** y **ReadAllText** que hacen la escritura y/o lectura completa de un archivo de texto, incluyen

apertura y cierre. Cuando se desea que sea entero son muy cómodas. Observa el siguiente ejemplo, es sencillo de entender:

```
string path = "myfile.txt";

try
{
    // 1. Write new file
    string initialContent = "Initial info.\n";
    File.WriteAllText(path, initialContent);

    // 2. Add a line
    string additionalContent = "Line added.\n";
    File.AppendAllText(path, additionalContent);

    // 3. Read a show the data
    string content = File.ReadAllText(path);
    Console.WriteLine("Content of file:");
    Console.WriteLine(content);
}
catch (Exception ex)
{
    Console.WriteLine($"Error: {ex.Message}");
}
```

Siempre usa bloques try-catch, al menos para manejar la siguientes excepciones de gestión de archivos clásicas.

FileInfo

Sí es necesario instanciarla pero nos va a ofrecer una serie de métodos añadidos de información sobre el archivo.

La información principal que ofrecerá sobre un archivo es su Nombre, su Extensión y el directorio en el que se encuentra.

Ejemplo :

```
FileInfo f;

f = new FileInfo("C:\\windows\\win.ini");
Console.WriteLine("Name: {0}\n"
            + "Extension: {1}\n"
            + "Directory: {2}\n"
            + "Length: {3} bytes\n\n"
            + "And the full name: {4}",
            f.Name, f.Extension, f.DirectoryName, f.Length, f.FullName);
```

Directorios: Las clases Directory y DirectoryInfo

Nuevamente la clase Directory nos da una serie de métodos genéricos y no es necesaria instanciarla (*Shared* o estática), sin embargo de DirectoryInfo sí hace falta crear un objeto para poder usarlo.

Veamos cuales son las principales posibilidades que nos da cada una:

Directory

GetCurrentDirectory: función que devuelve el directorio actual de trabajo. Inicialmente es en el que se encuentra el ejecutable de la aplicación.

SetCurrentDirectory(nombre): Establece un nuevo directorio de trabajo. Se puede usar indicadores relativos:

```
Directory.SetCurrentDirectory("..\")
```

CreateDirectory, Delete: Crea directorio y borra directorio vacío respectivamente**.**

DirectoryInfo

Instanciamos el objeto para tener más información sobre cierto directorio.

GetDirectories: Devuelve un **array** de **DirectoryInfo** que contiene todos los subdirectorios del directorio actual. Opcionalmente se le puede pasar un parámetro para filtrar los elementos a presentar.

Por ejemplo GetDirectories(“s*”) presenta sólo los directorios que comiecen por S.

GetDirectories(“DATOS??B”) Presenta los directorios que empiezan por datos, tiene luego dos caracteres cualesquiera y acaban por B.

GetFiles: Devuelve un **array** de **FileInfo** que contiene todos los archivos de ese subdirectorio. Se puede usar parámetros igual que en el caso anterior: GetFiles(“*.bmp”) obtiene todos los ficheros de extensión bmp

Tanto GetFiles como GetDirectories devuelve un array, por lo tanto para conocer la cantidad de directorios o ficheros se puede usar la propiedad Lenght de array.

Veamos el ejemplo siguiente para comprender mejor su funcionamiento.

```
DirectoryInfo d;
string systemRoot = Environment.GetEnvironmentVariable("SystemRoot");
Console.WriteLine($"{"Current directory:",-20}{Directory.GetCurrentDirectory()}");
Console.WriteLine($"{"Change to:",-20}{systemRoot}");

Directory.SetCurrentDirectory(systemRoot);

d = new DirectoryInfo(Directory.GetCurrentDirectory());

Console.WriteLine($"Subdirectories of {d.Name}");
foreach (DirectoryInfo dirs in d.GetDirectories())
{
    Console.WriteLine($"\t{dirs.Name}");
}

Console.WriteLine("Files:");
foreach (FileInfo f in d.GetFiles())
{
    Console.WriteLine($"\t{f.Name}");
}
```

El código es sencillo de entender. Se observa de nuevo el método para acceder a distintos directorios mediante variables de entorno.

Delegates (delegados)

Antes de entrar en la programación multihilo hay que dejar claro un concepto que se va a usar en muchas ocasiones y es el de delegado (delegate).

Es una variable que "guarda" una función en lugar de un dato. De esta forma podemos manejar una función como si fuera una variable pasándola como parámetro, asignándole distintas funciones, etc.

Esto en principio puede parecer raro pero en realidad no es así. Pensemos en el concepto de referencia a objeto: es una dirección de memoria donde se guardan datos. Pero a fin de cuenta, tanto los datos como el código ejecutable está en diversas posiciones de la misma memoria (**Arquitectura Von Neumann**) ¿Por qué no entonces tener un identificador que en lugar de apuntar a datos apunte a código ejecutable? Eso es un **delegado: una referencia (puntero) a una función.**

Esto será muy práctico, entre otras cosas, en el uso de programación multihilo y también para el uso y creación de eventos como se verá en la asignatura de Diseño de Interfaces.

Como ejemplo puedes ver **la asociación de eventos con sus funciones** en el código oculto por el IDE al crear **formularios** (un evento es similar a un delegado, se verá más adelante la diferencia).

También puedes ver en el siguiente ejemplo como a ciertas funciones de la clase **Array** se les puede pasar una función como parámetro para actuar sobre los datos.

```
public static void view(int grade)
{
    Console.ForegroundColor = grade >= 5 ? ConsoleColor.Green : ConsoleColor.Red;
    Console.WriteLine($"Student grade: {grade,3}.");
    Console.ResetColor();
}


public static bool pass(int num)
{
    return num >= 5;
}
```

```
static void Main(string[] args)
{
    int[] v = { 2, 2, 6, 7, 1, 10, 3 };
    Array.ForEach(v, view);
    int res = Array.FindIndex(v, pass);
    Console.WriteLine($"The first passing student is number {res+1} in the list.");
    Console.ReadKey();
}
```

Estas funciones de la clase Array se verán con más detenimiento en el apartado de expresiones lambda, pero se intuye lo que pueden hacer:

Array.ForEach(vector, función): Ejecuta una función que no devuelve nada sobre cada uno de los elementosdel vector. Obviamente el parámetro de la función es el mismo que el tipo base del vector: si el vector es de enteros, la función tienen que tener un parámetro entero para funcionar.

Array.FindIndex(vector, funcionBooleana): Recorre el vector y para cuando encuentre el primer elemento del vector que hace que la funcionBooleana sea true. En ese momento devuelve el índice del elemento que hace que la función sea true. La función debe tener el mismo tipo de parámetro que el tipo base del vector.

Esto tiene gran importancia en la programación multihilo ya que la base de esta programación es el hecho de que paso una función como parámetro al constructor de una clase denominada hilo (Thread). Es decir, voy a tener una construcción de este tipo:

```
Thread thread=new Thread(myFunction);
```

Con el detalle que el parámetro **myFunction** es realmente un método cualquiera del programa (con ciertas condiciones). Se dice que al constructor Thread se le pasa un delegado.

En programación clásica se trabajaba con los denominados punteros a funciones (o functions callbacks), es decir, si queríamos pasarle una función como parámetro a otra función se le pasaba realmente la dirección de memoria del código de la primera función. Un delegado viene a realizar algo similar.

Puede sonar raro al principio pero es totalmente lógico ya que el paso de parámetros es similar, no deja de ser una dirección de memoria donde hay un dato. Pues en este caso lo que se pasa como parámetro es una dirección de memoria dónde hay código para ejecutar.

De esta manera se puede cambiar de forma dinámica a lo largo de la ejecución de un programa las funciones que se ejecutan en determinados momentos. Además a un delegado se le puede pasar una lista de funciones de forma que al ser ejecutado ejecuta todas esas funciones.

La forma básica cómo se declara un delegado es:

```
delegate returnValue DelegateName(params)
```

Para usarlo se declara una "variable" del tipo delegado y se le pasa la función que deseemos al constructor.

Luego simplemente usando dicha variable con los parámetros requeridos se ejecutaría la función en cuestión.

Veamos todo esto con un ejemplo:

```
public delegate int Operation(int a, int b);

class Program
{
    static int Addition(int a, int b)
    {
        return a + b;
    }

    static int Substraction(int a, int b)
    {
        return a - b;
    }

    static void Main(string[] args)
    {
        Operation op = new Operation(Substraction); // Operation op = Substraction
        string res;
        int n1, n2;
        Console.WriteLine("Do you want to add or substract?(A/S)");
        res = Console.ReadLine().Trim().ToUpper();
        if (res == "A")
        {
            op = new Operation(Addition);      // Operation op = Addition;
        }
        Console.WriteLine("Input first operand");
        n1 = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("Input second operand");
        n2 = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("Result: {0}", op(n1, n2));
        Console.ReadKey();
    }
```

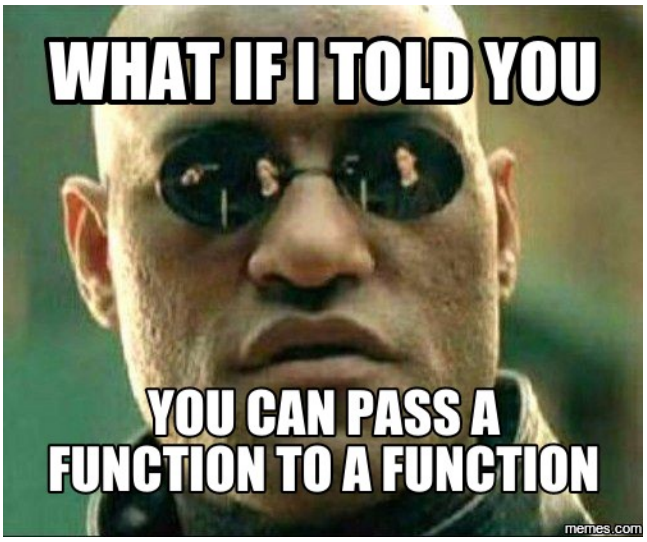
```
}
```

Es imprescindible cuando se crea un delegado que **las funciones que se le asocien cumpla su signatura**, es decir, que tanto el tipo del valor devuelto como la lista de parámetros coincida. En este caso tanto las funciones *suma* como *resta* devuelven un entero y se le pasan dos enteros. De esta forma simplemente llamando al constructor del delegado y pasando la función queda enlazada. Si no se cumple la signatura da error de compilación (Es por esto que los métodos asociados a eventos siempre tienen que cumplir la misma signatura).

Para ejecutar la función se puede hacer directamente con el nombre del delegado tal y como se ve en el ejemplo, o usar el método Invoke, ambas son válidas:

```
op(n1, n2)
op.Invoke(n1, n2)
```

Algunas características de uso de delegados



- Se puede asignar una función a un delegado sin necesidad de constructor (Ya desde C# 2.0):

  ```
  Operation op = Substraction;
  ```

- Se pueden añadir varias funciones a un delegado mediante el operador +=. Por ejemplo se podría hacer:

```
op += newFunction;
```

siempre que `newFunction` cumpla la signatura del delegado.

Prueba a cambiar las funciones Substraction y Addition de forma que muestren el resultado en lugar de devolver y añade las dos al delegado y luego ejecútalo.

Ojo, si se asigna con = se eliminan todas las funciones agregadas anteriormente.

- De forma análoga con -= se puede quitar una función de una lista de funciones de un delegado.

- Un delegado puede ser ser pasado como parámetro a otro método. De esta forma se pueden hacer métodos genéricos que procesan varios datos llamando a distintas funciones según la necesidad. Por ejemplo se puede realizar un método que recorra una colección y que sobre cada dato se ejecute el delegado que viene como parámetro: un método de visualización, o uno de búsqueda, etc.

  Por ejemplo una función para realizar distintas operaciones entre vectores:

```
static int[] VectorOperation(int[] v1, int[] v2, Operation op)
  {
      int[] v = null;
      if (v1 != null && v2 != null && v1.Length == v2.Length)
      {
          v=new int[v1.Length];
          for (int i = 0; i < v1.Length; i++)
          {
              v[i] = op(v1[i], v2[i]);
          }
      }
      return v;
  }
```

  Puedes usar para probar las funciones Addition o Substraction.

- Ver apéndice II para más información sobre delegados (Acceso a funciones de un delegado y declaración de delegados genéricos).

Expresiones Lambda

Una expresión lambda es una forma de realizar una función anónima, es decir, sin necesidad de definir un nombre para la misma. Es útil cuando se quieren hacer llamadas a métodos que usan delegados y se desea pasar una función breve a dicho delegado sin necesidad de declararlo.

Este tipo de funciones fueron definidas ya en los años 30 y existen lenguajes de programación que se basan totalmente en este tipo de funciones/expresiones como Haskell o Lisp. Son los denominados lenguajes de programación funcional (distinta a la programación imperativa de lenguajes como c# o Java).

Debido a ciertas ventajas de este tipo de programación diversos lenguajes actuales aceptan este paradigma a través de las funciones o expresiones lambda como c#, java y otros.

Una función Lambda tiene la siguiente sintaxis (a **=>** se le denomina **operador lambda**):

```
Parámetro/s => Operaciones
```

Veamos un ejemplo similar al que ya vimos en el apartado de delegados. Es el mismo programa pero sin las funciones usando en su lugar expresiones lambda:

```
public delegate int Operation(int a, int b);

class Program
{
    static void Main(string[] args)
    {
        Operation op = (a, b) => a - b;
        string res;
        int n1, n2;
        Console.WriteLine("Do you want to add or substract?(A/S)");
        res = Console.ReadLine().Trim().ToUpper();
        if (res == "A")
        {
            op = (a, b) => a + b;
        }
        Console.WriteLine("Input first operand");
        n1 = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("Input second operand");
        n2 = Convert.ToInt32(Console.ReadLine());
        Console.WriteLine("Result: {0}", op(n1, n2));
        Console.ReadKey();
```

```
    }
}
```

También se puede usar para operaciones rápidas. Por ejemplo el siguiente fragmento cuenta números pares en un vector:

```
int[] v = { 2, 4, 10, 43, 22, 32, 13, 6, 7 };
int even = v.Count(n => n % 2 ==0);
Console.WriteLine(even);
```

En este caso el método **Count** de los arrays admite como parámetro cualquier función con un parámetro entero y que devuelva una booleana. Se encarga de ejecutar esa función sobre cada valor del vector y si es true lo suma al contador que al final devuelve.

Algunos ejemplo más de paso de funciones como parámetro y uso de expresiones lambda para arrays:

```
int[] v = { -2, 9, 1, -7, 9, 4 };

/* Actúa sobre cada elemento de un array pasándoselo como parámetro a la función. No
permite la modificación de elementos del array. */
Array.ForEach(v, item => { Console.WriteLine(item*item); });
Console.WriteLine("========");

/* Muestra el índice de la primera aparición de un elemento de un array que haga que la
función devuelva true. Existe también Array.Exist que hace lo mismo pero simplemente
devuelve true o false si existe un elemento que haga que la función devuelve true */
Console.WriteLine(Array.Find(v, item => item < 0));
Console.WriteLine("========");

/* Similar a Find pero en este caso devuelve un vector con todas las ocurrencias que
hacen que sea true la función parámetro */
int[] negatives= Array.FindAll(v, item => item < 0);
Array.ForEach(negatives, item => { Console.WriteLine(item); });
```

Si no se dispone de parámetros se usan paréntesis vacíos, por ejemplo:

```
() => Console.WriteLine("Hello");
```

También se pueden usar para asociar eventos sencillos, por ejemplo si tenemos un formulario con un botón denominado Button1:

```
this.button1.Click +=
            (Sender, Ev) => MessageBox.Show("¡¡Welcome to the Lambda Expressions!!");
```

Los set y get en la definición de objetos también permiten realizar expresiones más breves:

```
private string telNumber;
public string TelNumber
{
    set => telNumber = value;
    get => "+34" + telNumber;
}
```

Características importantes:

- Los paréntesis de los parámetros son opcionales salvo que no haya parámetros, entonces son obligatorios

```
() =>"hello"  //obligatorios
x => x * x    //opcional, se puede poner (x) => x*x
```

- Si el compilador no puede deducir el tipo de parámetro este se puede especificar

```
object o = (a, b) => a + b; // Error
object o = (int a, int b) => a + b;
```

- Se puede hacer lambda con varias sentencia usando llaves:

```
n => {
       string s = n + " World";
       Console.WriteLine(s);
     };
```

- Algunos delegados habituales en .net que usan genéricos para la definición(para no tener que definirlos de nuevo):
  - *Action, Action<T>, Action<T1,T2>,…:* Indica una función que no devuelve nada y tiene los parámetros genéricos del tipo indicado en el operador diamante.
  - *Func<Tresult>, Func<T,Tresult>, Func<T1,T2,Tresult>,…:* Indica una función que devuelve un tipo TResult y tiene los parámetros del tipo indicado en el operador diamante.
  - *Predicate<T>:* Caso particular de Func limitado a un parámetro genérico

y que devuelve booleano. Se usa para establecer reglas (Si el parámetro cumple determinada condición devuelve true).

- Puedes ver más ejemplos en: https://www.c-sharpcorner.com/blogs/c-sharp-generic-delegates-func-action-and-predicate

Al igual que ya se habló en el tema correspondiente al switch, esta estructura también se puede aprovechar (desde C#9) de las funciones lambda para hacerlo aun más versátil. Puede ver ejemplo de uso en esta entrada:

https://stackoverflow.com/a/28949100/19091122

Más sobre expresiones lambda en:

https://msdn.microsoft.com/es-es/library/bb397687.aspx

Apéndice I: Archivos Binarios en C#

La clase FileStream

Esta clase hereda de la clase abstracta Stream que como ya se comentó sirve para el manejo de secuencias de bytes. Por tanto las clases que hereden de Stream nos servirán para manejar los archivos a nivel de byte.

En concreto FileStream nos permite realizar acciones de escritura y lectura byte a byte o en grupos de bytes. Por supuesto esto puede requerir futuras conversiones a tipos de datos buscados.

Además permite el **acceso aleatorio** a archivos.

Constructor*:* Nos permite indicarle el nombre del archivo a manejar y la forma de abrirlo (lectura, escritura, añadir,…). Hay varios sobrecargados, pero el que quizá más nos interese es el que admite **dos parámetros: Nombre del archivo y modo de apertura.** El segundo parámetro es un **enumerado FileMode** que puede tomar los valores:

Nombre	***Descripción***
`Append`	Abre el archivo si ya existe y se prepara para escribir al final del mismo, o crea un nuevo archivo si no existe. Cualquier intento de leer provoca un error e inicia una excepción ArgumentException.
`CreateNew`	Especifica que el sistema operativo debe crear un archivo nuevo. Si el archivo ya existe, se inicia una excepción IOException.
`Open`	Especifica que el sistema operativo debe abrir un archivo existente. Para lectura o escritura System.IO.FileNotFoundException se inicia si el archivo no existe.
`OpenOrCreate`	Especifica que el sistema operativo debe abrir un archivo si ya existe; en caso contrario, debe crearse uno nuevo.
`Truncate`	Especifica que el sistema operativo debe abrir un archivo existente. Una vez abierto, debe truncarse el archivo para que su tamaño sea de cero bytes. Por tanto vacía un archivo ya existente. Si el archivo no existe salta la excepción System.IO.FileNotFoundException
`Create`	Especifica que el sistema operativo debe crear un archivo nuevo. Si el archivo ya existe, se sobrescribirá. **System.IO.FileMode.Create** es equivalente a solicitar que se utilice **CreateNew** si no existe el archivo; si el archivo existe, es equivalente **Truncate**.

Se recomienda la instalación de un visor hexadecimal de archivos en el disco duro (p.ej. HxD) para poder hacer comprobaciones.

Algunos métodos:

WriteByte(valor): Escribe un byte en el archivo. Ejemplo:

```
string directory = Environment.GetEnvironmentVariable("userprofile");
FileStream fs;
using (fs = new FileStream(directory + "\\testfile.bin", FileMode.Create))
{
      fs.WriteByte(15);
      fs.WriteByte(20);
}

using (fs = new FileStream(directory + "\\testfile.bin", FileMode.Append))
{
      fs.WriteByte(200);
      fs.WriteByte(255);
}
```

ReadByte(): Función que devuelve un byte (convertido a integer) del archivo indicado. El archivo debe estar abierto en alguna configuración de lectura. Devuelve -1 si se llega al final del archivo (es el -1 el motivo por el que devuelve un tipo int en vez de byte, si no podría devolver tipo byte).

Ejemplos: Dos formas de leer todos los bytes de un archivo y mostrarlos en pantalla.

```
// Mediante bucle for y Length
string directory = Environment.GetEnvironmentVariable("homepath");
FileStream fs;
using (fs = new FileStream(directory + "\\testfile.bin", FileMode.Open))
{
    for (long i = 0; i < fs.Length; i++)
    {
        Console.Write("{0} ", fs.ReadByte());
    }
}

// Mediante detección de fin de archivo (-1)
using (fs = new FileStream(directory + "\\testfile.bin", FileMode.Open))
{
    int valor = fs.ReadByte();
    while (valor != -1)
    {
        Console.Write("{0} ", valor.ToString());
        valor = fs.ReadByte();
    }
}
```

Seek(desplazamiento, origen): Sitúa el cursor de lectura del archivo en la posición deseada. Nos permite el acceso aleatorio al archivo.

Enumeración **SeekOrigin**:

Begin	Especifica el comienzo de una secuencia.
Current	Especifica lo posición actual dentro de la secuencia.
End	Especifica el final de una secuencia.

En el caso de usar como origen Current o End, se pueden usar valores negativos en el desplazamiento.

Aquí un ejemplo sencillo de uso:

```
using (fs = new FileStream(directory + "\\testfile.bin", FileMode.Append))
{
      fs.WriteByte(200);
      fs.Seek(16, SeekOrigin.Begin);
      fs.WriteByte(255);
}
```

Pero en un caso real esto suele ser necesario para gestionar archivos más complejos. Como ejemplo podemos leer parte de la información de un **archivo BMP**. Para ello, tenemos que tener acceso al formato de este tipo de imágenes. Este formato son estándares de la industria y los más habituales suelen estar disponibles de forma pública.

Puedes ver un resumen sencillo en: https://es.wikipedia.org/wiki/Windows_bitmap

Y más completo en: https://en.wikipedia.org/wiki/BMP_file_format

En un BMP de Windows, **los dos primeros** bytes (0 y 1) son los ASCII correspondientes a las letras **B** y **M** (un byte por letra).

Los bytes siguientes (2, 3, 4 y 5) son el tamaño del archivo (**iojo, en little endian!**) y del 10 al 13 se indica donde empieza la información de los pixels de la imagen. Veamos una tabla con datos a partir del byte 14 (0x0E) que es la que usaremos principalmente para este ejemplo:

Offset (hex)	Offset (dec)	Size (bytes)	Windows `BITMAPINFOHEADER`[2]
0E	14	4	the size of this header, in bytes (40)
12	18	4	the bitmap width in pixels (signed integer)
16	22	4	the bitmap height in pixels (signed integer)
1A	26	2	the number of color planes (must be 1)
1C	28	2	the number of bits per pixel, which is the color depth of the image. Typical values are 1, 4, 8, 16, 24 and 32.
1E	30	4	the compression method being used. See the next table for a list of possible values
22	34	4	the image size. This is the size of the raw bitmap data; a dummy 0 can be given for BI_RGB bitmaps.
26	38	4	the horizontal resolution of the image. (pixel per metre, signed integer)
2A	42	4	the vertical resolution of the image. (pixel per metre, signed integer)
2E	46	4	the number of colors in the color palette, or 0 to default to 2^n
32	50	4	the number of important colors used, or 0 when every color is important; generally ignored

Con la información previa podemos por ejemplo saber si un archivo es un BMP (la comprobación es muy simple, habría que hacerla algo más exhaustiva) y además cuantos bits por pixel usa:

```
using (FileStream fs = new FileStream(BmpFile, FileMode.Open))
{
    string format = ((char)fs.ReadByte()).ToString() +
                    ((char)fs.ReadByte()).ToString();

    if (format == "BM")
    {
        fs.Seek(0x1c, SeekOrigin.Begin);
        Console.WriteLine($"The image is a {fs.ReadByte()} bits Windows BMP");
    }
    else
    {
        Console.WriteLine("The file isn't a Windows BMP");
    }
}
```

Las clases BinaryReader y BinaryWriter

Son clases similares a FileStream pero que permite el acceso al archivo de un tipo fijo de dato.

Por ejemplo BinaryWriter puede hacer Write de caracteres, enteros, reales etc…

Y BinaryReader dispone de los métodos *ReadChar, ReadInteger, ReadBoolean*, etc.

A diferencia de las anteriores clases vistas, estas no se pueden inicializar a partir del nombre de un archivo, si no que hay que hacerlo a través de un FileStream. Por ejemplo:

```
BinaryWriter bw;
bw = new BinaryWriter(new FileStream(directory+"\\testfile.bin",
FileMode.CreateNew));


BinaryReader br;
br = new BinaryReader(new FileStream(directory+"\\testfile.bin", FileMode.Open));
```

Una **diferencia importante** entre el BinaryWriter y el BinaryReader es que el primero dispone directamente del método **seek** tal y como lo vimos en el FileStream mientras el segundo no. Pero esto no es problema ya que disponemos del seek del FileStream asociado.

```
fs = New FileStream(name, FileMode.Open)
br = New BinaryReader(fs);

fs.Seek(pos, SeekOrigin.Begin);

o

br.BaseStream.Seek(pos, SeekOrigin.Begin);
```

Como ejemplo vemos como usar un BinaryReader en la lectura de algunos datos de un archivo BMP (Para entenderlo bien debes ver la tabla de formato BMP presentada en el punto anterior):

```
using (BinaryReader br = new BinaryReader(new FileStream(BmpFile, FileMode.Open)))
{
    string format = new string(br.ReadChars(2));

    if (format == "BM")
    {
        br.BaseStream.Seek(0x1c, SeekOrigin.Begin);
        Console.WriteLine($"The image is a {br.ReadInt16()} bits Windows BMP");
        br.BaseStream.Seek(0x16, SeekOrigin.Begin);
        Console.WriteLine($"The image height is {br.ReadInt32()} pixels");
    }
    else
    {
        Console.WriteLine("The file isn't a Windows BMP");
    }
}
```

BinaryWriter y BinaryReader "a medida"

Cuando queremos hacer la lectura o escritura sobre algún tipo de dato que hemos creado nosotros como puede ser una clase, lo mejor es heredar las clases BinaryWriter y BinaryReader y ampliarlas con funciones para la gestión de la clase que queremos tratar.

Lo vemos con un ejemplo:

```
    public class Person
    {
        public string Name { set; get; }
        public int Age { set; get; }
    }


    class BinWriterPerson : BinaryWriter
    {
        public BinWriterPerson(Stream str) : base(str)
        {
        }

        public void Write(Person p) //Overload
        {
            base.Write(p.Name);
            base.Write(p.Age);
        }
    }
```

```
class BinReaderPerson : BinaryReader
{
    public BinReaderPerson(Stream str) : base(str)
    {
    }

    public Person ReadPerson()
    {
        Person p= new Person();
        p.Name = base.ReadString();
        p.Age = base.ReadInt32();
        return p;
    }
}
```

Posteriormente en el programa principal o en un método deseado de cualquier clase se pueden escribir las operaciones de lectura y escritura de datos. En la variable **directorio** lee la variable de entorno deseada.

```
// Write
Person person = new Person();
BinWriterPerson bwp;
string directory = Environment.GetEnvironmentVariable("homepath");

person.Name = "Mr. Potato";
person.Age = 33;

bwp = new BinWriterPerson(new FileStream(directory + "\\people.dat",
FileMode.CreateNew));
bwp.Write(person);
bwp.Dispose(); //Dipose implies close


// Read a show data
BinReaderPerson brp;
brp = new BinReaderPerson(new FileStream(directory + "\\people.dat",
FileMode.Open));
person = brp.ReadPerson();
brp.Dispose();

Console.WriteLine(person.Name + " " + person.Age);
```

Se puede ver con un programa como HxD como queda el "formato" a medida que has creado para tu archivo de personas.

Apéndice II: Más sobre delegados

Acceso a las funciones de un delegado

Mediante la función *GetInvocationList()* se puede acceder por separado a la lista de funciones mediante indexación en un delegado. Si retomamos el ejemplo visto en el tema y añadimos tanto *Addition* como *Substraction* al delegado op podemos ejecutar las siguientes líneas para que nos devuelva el valor de cada función por separado:

```
Console.WriteLine("Result: {0}", ((Operation)op.GetInvocationList()[0])(4, 6));
Console.WriteLine("Result: {0}", ((Operation)op.GetInvocationList()[1])(4, 6));
```

Delegados genéricos

Existe la posibilidad de realizar un delegado genérico mediante la construcción:

```
public delegate void GenericDelegate<T>(T arg);
```

Es decir, un delegado que acepta cualquier función de un único parámetro.

Posteriormente para usarlos se establecerían de la siguiente forma:

```
GenericDelegate<int> dg1 = new GenericDelegate<int>(function1);
GenericDelegate<double> dg2 = new GenericDelegate<double>(function2);
```

Las funciones tendría que tener la siguiente forma:

```
public static void function1(int n) { … }

public static void function2(double d) { … }
```

La limitación de que solo se le pueda pasar un parámetro se obvia con varios genéricos <T1,T2,...> (T1 arg1, T2 arg2,...)